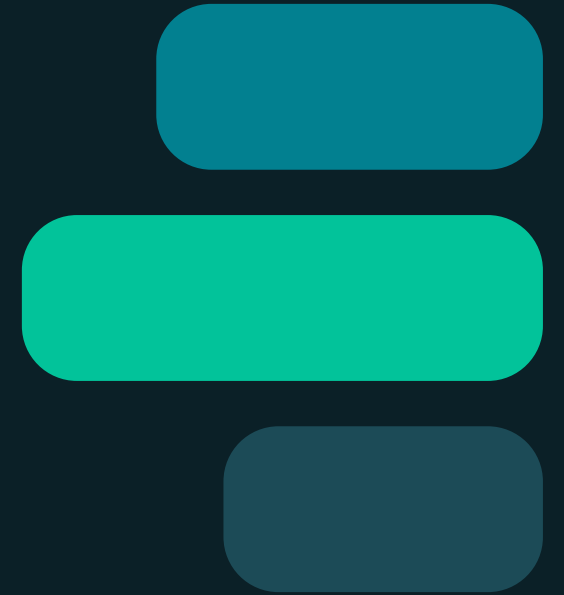


Building Chatbots with Python & Dialogflow

From simple if-else rules to a cloud-deployed NLU agent

Rule-based • Flow-based • Dialogflow • Deploy



What we cover in two days

DAY 3

Build it yourself in Python

- Rule-based chatbot (keywords)
- Pattern matching with regex
- Flow-based design: states & slots
- Demo: PizzaBot order flow
- Lab: build a CampusBot

DAY 4

Let Dialogflow do the thinking

- Why rules break - NLU basics
- Agent, Intent, Entity, Context
- Build the agent in the console
- Fulfillment webhook (Flask)
- Deploy to Cloud Run + channels

Two families of chatbots

Rule-Based

IF this word THEN that reply

STRENGTHS

- Fully predictable
- No training data
- Runs anywhere, free

LIMITS

- Breaks on new wording
- Grows messy after ~50 rules

NLU-Based

Understands INTENT, not words

STRENGTHS

- Handles unseen phrasing
- Extracts entities automatically
- Multi-language ready

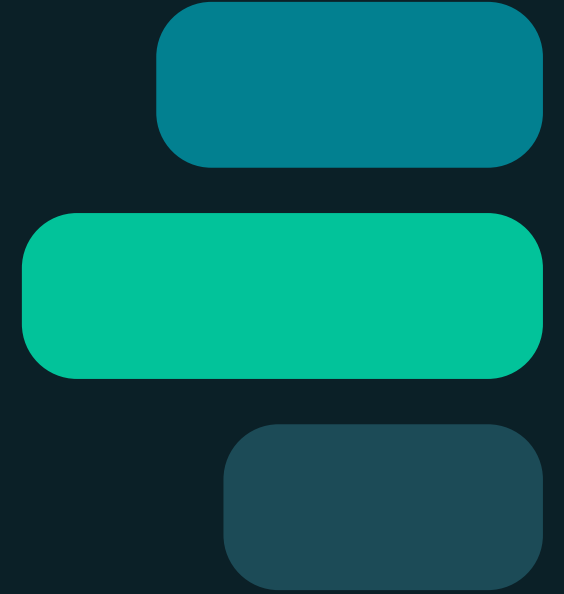
LIMITS

- Needs a platform + training
- Confidence, not certainty

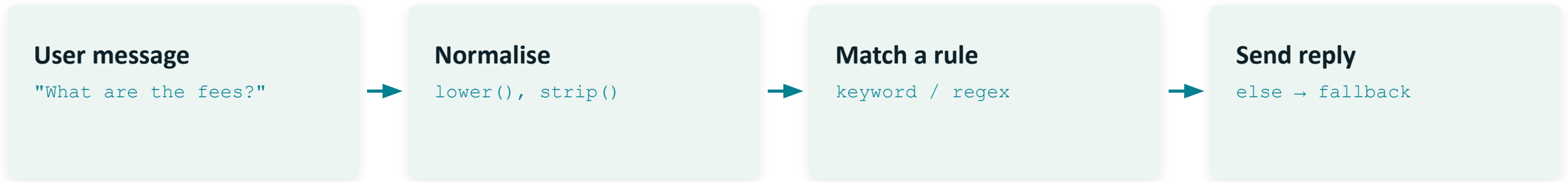
Rule of thumb: closed domain with few questions → rules. Open wording, many variations, real users → Dialogflow.

Rule-Based & Flow-Based Bots

Pure Python • no libraries • 3 working demos



How a rule-based chatbot works



The 3 pieces of every rule bot

- 1 Rule store** — dictionary / list of patterns
- 2 Matcher** — the function that picks a rule
- 3 Fallback** — what to say when nothing matches

Where rules break

"fees" → matched

"How much do I have to pay?" → no match

"cost of the semester?" → no match

Same intent, different words. This is exactly the gap Dialogflow fills on Day 2.

Rule-based bot with keywords

```
RULES = {
    ("hi", "hello"): "Hello! I am CampusBot.",
    ("fee", "fees"): "Semester fee is Rs. 45,000.",
    ("timing", "hours"): "Office: Mon-Fri, 9 AM - 5 PM.",
    ("bye", "exit"): "Goodbye! Happy learning.",
}

FALLBACK = "Sorry, try: fees / timings / courses."

def get_response(message):
    text = message.lower()
    for keywords, reply in RULES.items():
        if any(word in text for word in keywords):
            return reply
    return FALLBACK

while True:
    user = input("You: ")
    print("Bot:", get_response(user))
```

Output

```
You: hello
Bot: Hello! I am CampusBot.
You: what about fees?
Bot: Semester fee is Rs. 45,000.
You: aaaa
Bot: Sorry, try: fees / ...
```

Teaching points

- lower() before matching
- any() = OR across keywords
- order matters: first hit wins
- never skip the fallback

Patterns that also capture data

```
import re

PATTERNS = [
    (r"\b(hi|hello)\b",      "Hi there! I am ShopBot."),
    (r"my name is ([a-z]+)",  "Nice to meet you, {0}!"),
    (r"track order (\d+)",    "Order #{0} arrives in 2 days."),
    (r"\b(refund|return)\b",  "Returns accepted within 7 days."),
]

def get_response(msg):
    text = msg.lower()
    for pattern, reply in PATTERNS:
        match = re.search(pattern, text)
        if match:
            # {0} is filled with the captured group
            return reply.format(*match.groups())
    return "Could you rephrase that?"
```

Output

```
You: my name is ravi
Bot: Nice to meet you, ravi!
You: track order 4521
Bot: Order #4521 arrives in 2 days.
```

Regex cheat sheet

<code>\b</code>	word boundary
<code>(...)</code>	capture group
<code>\d+</code>	one or more digits
<code>a b</code>	a OR b
<code>(?:...)</code>	group, don't capture

Flow-based chatbot design

The problem

A keyword bot answers one question and forgets everything. It cannot take an order, book a seat, or fill a form.

The fix

Give the bot a STATE (where we are) and SLOTS (what we have collected). One question per state.

Four design steps before you write any code

1

List the slots

What data must you collect? size, topping, address

2

Order the questions

One slot per state, most important first

3

Validate each answer

Wrong input → stay in the state, re-ask

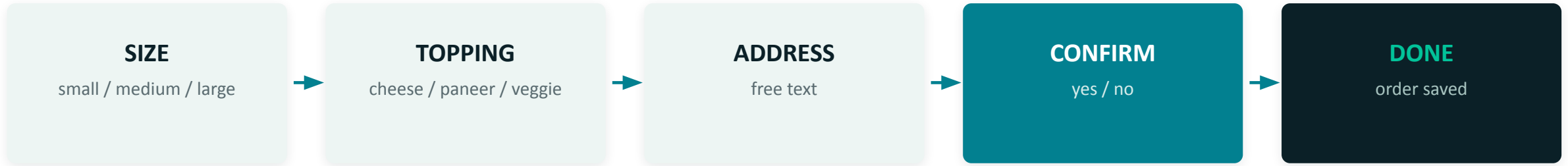
4

Plan the escapes

cancel / restart / confirm before committing

EXAMPLE

PizzaBot conversation flow



The bot greets first, then walks these states one by one — one question per state.

Invalid answer

User: "huge"

Bot: "Please choose small, medium or large."

State does NOT change — the bot simply asks again.

Escape hatches

- cancel / restart → reset state
- confirm before saving the order
- allow going back one step

Slots after the flow

```
order = {  
  "size":    "large",  
  "topping": "paneer",  
  "address": "12 MG Road"  
}
```

Flow bot as a state machine

```
class PizzaBot:
    def __init__(self):
        self.state = "SIZE"      # where we are
        self.order = {}          # what we collected

    def reply(self, text):
        text = text.lower().strip()

        if self.state == "SIZE":
            if text not in ("small", "medium", "large"):
                return "Please choose small, medium or large."
            self.order["size"] = text
            self.state = "TOPPING"
            return "Which topping? (cheese/paneer/veggie)"

        if self.state == "TOPPING":
            self.order["topping"] = text
            self.state = "DONE"
            return f"Confirmed: {self.order}"
```

Output

```
You: huge
Bot: Please choose small,
     medium or large.
You: large
Bot: Which topping?
You: paneer
Bot: Confirmed: {...}
```

Why a class?

- Each user gets their own bot object = their own state
- On a website, store the object per session id
- Full version uses one method per state

Your turn: build a CampusBot

Task

- 1 Write 6 keyword rules: fees, timings, courses, hostel, exam, contact
- 2 Add 2 regex rules that capture data (roll number, subject name)
- 3 Build a 3-state flow: BOOK APPOINTMENT → date → time → confirm
- 4 Handle invalid input at every state and add a 'cancel' command
- 5 Print the collected slots at the end as a dictionary

Start from `1_keyword_bot.py` and `3_flow_bot.py` in the code folder.

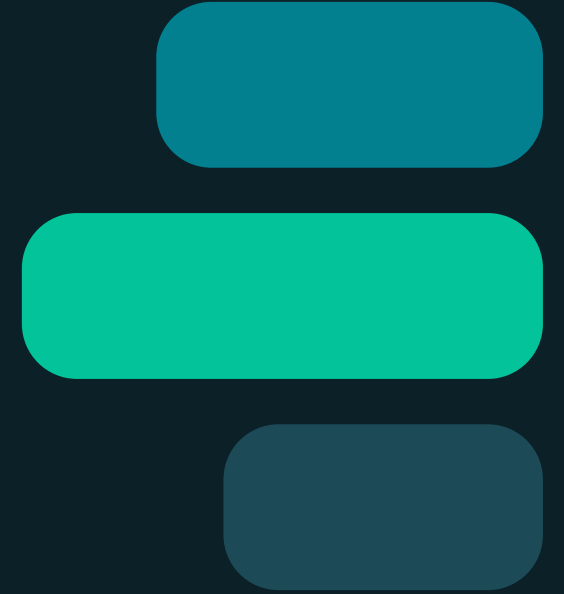
Stretch goals

- Log every conversation to a .txt file
- Add a synonym map so 'cost' also matches 'fees'
- Support 'go back' to the previous state
- Wrap it in Flask and chat through a browser page
- Count how many rules you needed - keep that number for tomorrow

DAY 4

Dialogflow & Deployment

Build an NLU agent • webhook in Flask • ship it to the cloud



Why move to Dialogflow?

Yesterday's bot

"I want a large paneer pizza"

→ no keyword matched

→ you write yet another rule

Every new phrasing = new code, new deploy.

With Dialogflow

"I want a large paneer pizza"

→ intent: order.pizza (0.94)

→ size: large, topping: paneer

You give ~10 example phrases; the model generalises to the rest.

What you get out of the box

NLU model

trained from your examples

Entity extraction

dates, numbers, custom lists

Channels

web, Telegram, WhatsApp

Analytics

what users actually ask

Six concepts that run Dialogflow

1

Agent

Your whole chatbot project. One agent per bot.

2

Intent

A goal the user has: order.pizza, order.status

3

Training phrases

10-15 sample sentences per intent

4

Entity

Data pulled out: @size, @topping, @sys.date

5

Context

Memory that links one intent to the next

6

Fulfillment

Your webhook — the real business logic

Flow of one message: user says something → Dialogflow picks the INTENT → extracts ENTITIES → calls your FULFILLMENT → replies

Build the PizzaBot agent in the console

1

Create the agent

dialogflow.cloud.google.com → Create Agent → name it PizzaBot, pick a language and time zone

2

Create entities

@size = small, medium, large · @topping = cheese, paneer, veggie (add synonyms!)

3

Create the intent

Name it order.pizza and add 10-15 training phrases

4

Annotate the phrases

Select 'large' → tag as @size. Dialogflow now extracts it automatically

5

Mark parameters required

Add prompts: 'What size?' — this gives you slot filling for free

6

Test in the right panel

Type a phrase you never trained and check intent + confidence

EXAMPLE

Anatomy of the order.pizza intent

1 · Training phrases you type in

```
I want a large paneer pizza  
get me a medium cheese pizza  
order pizza  
can I have a small veggie one
```

2 · Entities you annotate

```
large    → @size  
paneer   → @topping  
tomorrow → @sys.date   (built-in)  
9876543210 → @sys.phone-number
```

3 · What Dialogflow sends to your webhook

```
{ "queryResult": {  
  "queryText": "I want a large paneer pizza",  
  "intent": { "displayName": "order.pizza" },  
  "parameters": { "size": "large", "topping": "paneer" },  
  "intentDetectionConfidence": 0.94 } }
```


Fulfillment webhook in Flask

```
from flask import Flask, request, jsonify
app = Flask(__name__)

@app.route("/webhook", methods=["POST"])
def webhook():
    body = request.get_json(force=True)
    intent = body["queryResult"]["intent"]["displayName"]
    params = body["queryResult"]["parameters"]

    if intent == "order.pizza":
        size = params.get("size", "medium")
        topping = params.get("topping", "cheese")
        price = {"small":199,"medium":299,"large":399}[size]
        # real project: save the order to a database here
        reply = f"{size} {topping} pizza confirmed. Rs.{price}"
    else:
        reply = "Sorry, I cannot handle that yet."

    return jsonify({"fulfillmentText": reply})
```

Connect it

```
python 4_dialogflow_webhook.py
ngrok http 5000

copy the https URL
paste + /webhook into
Dialogflow > Fulfillment
```

Remember

- HTTPS only — no plain HTTP
- Must reply within 5 seconds
- Enable fulfillment on the intent
- Key must be fulfillmentText

Where can the bot actually live?

1

Web Demo

~0 min

One-click page from the console. Perfect for a college demo or viva.

2

Messaging apps

~10 min

Telegram, Messenger, Slack, WhatsApp — paste a token in Integrations.

3

Your own website

~30 min

Embed the Messenger iframe, or call detectIntent from your backend.

4

Custom app / IVR

~1 hr

Python, Node or Java client SDK. Full control over the UI.

Two separate deployments: the AGENT lives in Google's cloud already — only your FULFILLMENT WEBHOOK needs hosting.

Take the webhook live on Cloud Run

ngrok · development only

Instant public HTTPS URL to your laptop. URL changes on every restart and dies when you close the terminal.

Cloud Run · production

Permanent HTTPS URL, auto-scales to zero when idle, free tier covers a class project. Same Google project as the agent.

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY 4_dialogflow_webhook.py .
CMD exec gunicorn --bind :$PORT \
    4_dialogflow_webhook:app
```

Deploy in one command

```
gcloud run deploy pizza-webhook \
    --source . \
    --region asia-south1 \
    --allow-unauthenticated

# then paste the printed URL + /webhook
# into Dialogflow > Fulfillment
```

Put the bot inside your own app

```
from google.cloud import dialogflow

def detect_intent(text, session_id):
    client = dialogflow.SessionsClient()
    session = client.session_path(PROJECT_ID, session_id)

    query = dialogflow.QueryInput(
        text=dialogflow.TextInput(text=text,
                                   language_code="en"))

    res = client.detect_intent(session=session,
                               query_input=query)
```

Setup

```
pip install google-cloud-dialogflow
export GOOGLE_APPLICATION_CREDENTIALS="key.json" # service account
```

Go-live checklist

- One session id per user — never share it
- Webhook returns HTTP 200 under 5 seconds
- Service-account key stays out of Git
- Write a Default Fallback reply that helps
- Add small talk + a 'talk to human' intent
- Review the History tab weekly and retrain

WRAP UP

What you can build now

1

Rules

keyword & regex bots in ~30 lines of Python

2

Flows

state machines that collect slots and validate

3

Dialogflow

intents, entities, training phrases, slot filling

4

Deploy

Flask webhook on Cloud Run + any channel